Jev in a Multi-Step Workflow: Predicting Failures Is Not Knowing Which Reviews Pay

Thejas P Rao
thejasrao262003@gmail.com

Tushar M
tusharm3040@gmail.com

Dr Prajwala T R
PES University, Bengaluru, India
prajwalatr@pes.edu

***Abstract*—We set out to test a common design for AI workflows: chain a model's typed decisions, and use the model's own confidence to decide which cases a person should check. Our test bed is a bug-triage workflow built from the three primitives of Jev, TypeSafe's typed decision model (a categorical `choice`, a binary `noul` and an ordinal `score`). After exploratory work on 442 Mozilla and Linux kernel bugs, we ran pre-registered tests on 500 new bugs, four further models and a separate Kubernetes workflow. We count a failure when the workflow's action differs from a target built from tracker labels, a target we have not yet validated against human judgment. The models' confidence turned out to be a poor guide. A frozen estimator that reads only what the workflow decided ranked failures far better (AUROC 0.76 against 0.55 on the new bugs). It did so for all five models and again on Kubernetes, where only Jev and OpenJev handled every step competently. Most of its signal is the past error rate of the chosen action, which shifts when the mix of labels changes. Better ranking did not lower cost: under our pre-registered cost matrix, with reviewers assumed correct, no 10% review policy beat doing no review at all.**

***Index Terms*—uncertainty quantification, calibration, structured outputs, decision workflows, selective prediction, escalation, label shift, human review, LLM evaluation, bug triage**

I. Introduction

This project started as a study of compositional uncertainty. Typed decision models such as Jev, TypeSafe's model, return a label, a yes/no or a rating together with a probability or confidence. We wanted to know whether the confidence reported at each step of a workflow built from such answers could be combined into one signal for when to hand a case to a person. If every step says how sure it is, the workflow ought to know when it is likely to be wrong.

Our first analyses, on real Mozilla and Linux kernel bugs, did not bear this out. Jev's three primitives report uncertainty in different forms: `choice` and `score` return a distribution with a scalar confidence, while `noul` returns a single probability. Composed confidence ranked the workflow's failures no better than chance (Section IV). What did rank them was simpler, namely which action the workflow had chosen. That observation reshaped the study. We froze a predictor built on the decisions, collected new data and organised the rest of the study around three research questions. RQ1 asks which signals rank workflow failures, RQ2 whether they stay useful when the label mix shifts between development and deployment, and RQ3 whether ranking failures helps choose the cases worth reviewing.

Jev has been benchmarked on single tasks [1]–[3], and uncertainty along agent trajectories has been studied separately [4]. Neither line of work follows structurally different typed decisions through a chain into an escalation policy. TypeSafe describes Jev's confidence as one reasonable summary of a distribution and advises setting thresholds on one's own data [5]. So rather than test a calibration claim, this paper looks at how the numbers behave once the primitives are chained.

The study's value lies in how it was run rather than in a new technique; Park et al. [6] already argue in general that error risk and review value differ. We define failure on the action a workflow takes, and score a fixed review budget against a frozen cost matrix and an oracle bound. Every confirmatory test was pre-registered with the estimators frozen by hash, and the main result was replicated across five models and a second, chronologically split workflow.

II. Related work

Jev and typed decisions. Deußer et al. benchmarked Jev zero-shot on 37 datasets and found well-calibrated `choice` probabilities. Its binary probabilities ranked well but sat poorly against a 0.5 threshold [1]. A calibration audit found that calibration changed with the question form, and that one fitted temperature removed most miscalibration [2]. A study of political-science replications found Jev's calibration advantage inconsistent [3]. Open instruction-tuned models can produce Jev-like decisions from answer-label logits [7], which is the readout we use for the general models. All of this work evaluates single requests, and most of it is from 2026 and not yet peer reviewed.

Calibration, selection and deferral. Post-hoc calibration [8], [9], selective prediction [10], [11] and learning to defer [12], [13] all concern one decision. Bansal et al. show that the most accurate model need not make the best human–AI team [14]. Park et al. argue that error risk and review value are different quantities under a review budget [6]. Our cost result is a case of the second point in a dependent workflow.

Label shift and composition. Posteriors can be adjusted to new class priors [15], and the priors can be estimated from a black-box predictor's confusion matrix (BBSE) [16]. Product rules for combining classifiers assume independent, comparable scores [17]. Conformal methods have been composed over pipeline stages [18], and step-level calibration does not guarantee trajectory-level calibration for agents [4]. LLM raters compress ordinal scales toward the middle [19].

Cascades and software reliability. Cascades route a single query through model tiers and decide when to defer [20]–[22]. Reliability models treat error propagation between components as an architectural property that decides where recovery belongs [23], and our workflow is a small instance of that problem, with human escalation as the recovery step. Bug severity prediction is itself a long-studied triage task [24].

Our literature search was web-based and did not systematically cover reliability engineering or human–computer interaction. Within it, we found no prior work that tests which signals predict failure in a chain of structurally different typed decisions, on real labels and held-out data. Nor did we find work that asks whether those signals find the reviews that pay.

III. Methodology

Data and workflow. Figure 1 shows the pipeline. For development we use 202 Mozilla and 240 Linux kernel networking and storage bugs, sampled in equal quotas per subsystem and severity level. The held-out set has 500 newer bugs (250 per tracker) at each tracker's natural label mix. The replication uses 1,082 Kubernetes issues. Texts are privacy-scrubbed and truncated to 2,000 characters. Node 1 (`choice`) routes a report to a subsystem, node 2 (`noul`) asks whether it is a regression, and node 3 (`score`) rates severity on three levels. Each node sees the evidence plus the earlier decisions. Every node is called three times, and the answers are reduced to one decision by modal vote (for `noul`, the mean probability is thresholded at 0.5). All Jev calls resolved to `jev-1.13.0`. The open models are deterministic, so their three answers are identical and repeat instability plays no role for them.

Action, target and failure. The rule F maps an unclear c_2 to *request evidence* and $c_3 = 2$ to *escalate*. It maps $c_3 = 1$ with $c_2 = 1$ to *defer*, and anything else to *accept*. The target is the same rule applied to the reference labels, $a^* = F(c_2^*, c_3^*)$. A failure is $y = \mathbf{1}[a \neq a^*]$. F is our construct, so every failure result is conditional on it.

Failure signals. We compare three kinds of signal. The first is the model's final-step confidence, alone or composed across nodes. The second is the action-rule score $r(a) = (e(a) + \bar{r})/(n(a) + 1)$, the smoothed error rate of the chosen action on development data. The third is a logistic estimator on the decisions (E_{dec}), fitted on development data and frozen. In Section VI we add a post hoc baseline that passes the model's own probabilities through F. Because $r(a)$ takes one value per action, its AUROC involves many ties. In the cost test, ties at the budget boundary are resolved as an exact expectation over a random order.

Evaluation. AUROCs use the Mann–Whitney estimator with 95% bootstrap intervals over bugs ($B = 2{,}000$). Confirmatory tests are one-sided paired bootstrap tests with Holm correction. Costs come from a matrix frozen in advance. A wrong *accept* costs 6–10 and any other mismatch 1, and each escalation adds 1 for the reviewer's time.

Novelty. Table I lists the elements of the design and what each lets us conclude. The contribution lies in combining them.

TABLE I
Methodological elements and what each allows us to conclude.

Element	What it allows
Workflow-level failure	Failure is defined on the action taken, so every step-level signal is tested as a predictor of it
Ranking vs. cost	A fixed 10% review budget, a frozen cost matrix and an oracle bound separate "ranks failures" from "reduces cost"
Shift by design	Development and deployment label mixes differ, which exposes the label-prior dependence of error rates
Pre-registration	Hash-frozen estimators and dated amendments separate confirmatory from exploratory claims
Replication	Five models behind one interface and an independent Kubernetes workflow

TABLE II
Node agreement with reference labels on the development bugs (Wilson 95% intervals). Baseline: the most frequent label.

	Mozilla		Kernel	
Node	Agreement	Base	Agreement	Base
`choice`	0.89 [0.83, 0.92]	0.59	0.975 [0.95, 0.99]	0.50
`noul`	0.80 [0.74, 0.85]	0.74	0.725 [0.67, 0.78]	0.81
`score`	0.53 [0.46, 0.60]	0.40	0.44 [0.38, 0.50]	0.33

IV. Exploratory findings on the development bugs

The three primitives' confidences behave very differently. `choice` confidence is near 1.0 for most bugs (85% of Mozilla and 96% of kernel bugs are at ≥ 0.90). Yet it still discriminates its own errors (AUROC 0.90 [0.83, 0.96] on Mozilla, 0.98 [0.95, 1.00] on the kernel). `noul`'s confidence-like value carries no usable information (AUROC 0.53 [0.42, 0.65] and 0.41 [0.32, 0.50]). `score`'s confidence is about the same at every true level (AUROC 0.58 [0.50, 0.66] and 0.57 [0.50, 0.64]). Fixed thresholds tuned on constructed data did not transfer. On real text, a `choice` trigger at 0.90 fired on 85% and 96% of bugs. A product cut escalated 88% and 78% of bugs against targets of 41% and 42%.

The answers themselves are uneven (Table II). `choice` routes well above baseline on both trackers. `noul`'s errors change direction. On Mozilla it mostly misses regressions (23 misses vs. 17 false alarms). On the kernel it over-flags them (63 false alarms vs. 3 misses). `score` compresses severity toward the middle level (Fig. 2). Most minor bugs are rated higher than their label (79% on Mozilla, 75% on the kernel), and many serious bugs lower (43% and 56%). Using Mozilla's own severity definitions moved agreement only from 0.53 to 0.56, so the compression does not come from our mapping.

A naive comparison of composition rules looks favourable to decisions. On Mozilla, the decision-based action beat a confidence-product rule by 0.17 [0.08, 0.27] in the probability of the right action (exact McNemar $p = 0.0004$). But the confidence rule never reads the decisions, while the target escalates

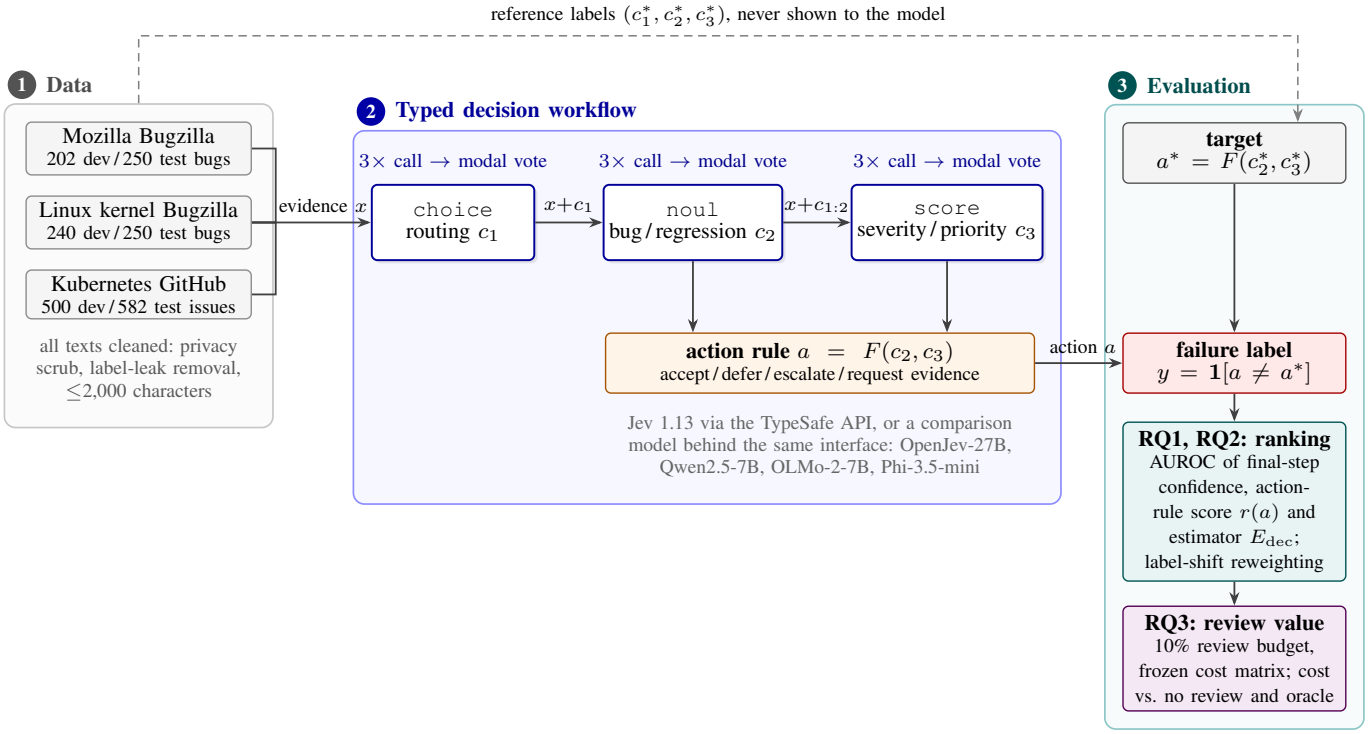


Fig. 1. Study architecture. Reports from three public trackers are cleaned and stripped of label leaks, then pass through three typed decisions, each called three times and reduced by modal vote. An action rule F maps the second and third decisions to an action. Reference labels, never shown to the model, give the target a^* through the same rule. A failure is $a \neq a^*$.

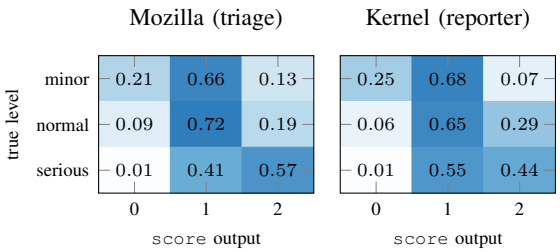


Fig. 2. Row-normalized confusion of `score` against reference severity. On both trackers the middle column absorbs mass from both extremes.

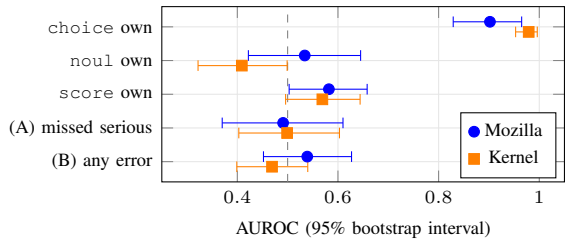


Fig. 3. Confidence AUROCs on the development bugs. The dashed line marks chance. Only `choice`'s confidence discriminates its own errors. Composed confidence is at chance for both escalation targets: (A) missed serious bugs and (B) any wrong action.

on the severity decision, so that comparison is confounded. The fairer test starts from the decision-based action and adds escalations ranked by low composed confidence. Among bugs the rule did not escalate, composed confidence ranked missed serious bugs at chance (AUROC 0.49 [0.37, 0.61] and 0.50 [0.40, 0.60]). Across all bugs, it ranked wrong actions at chance as well (0.54 [0.45, 0.63] and 0.47 [0.40, 0.54]). At a budget of 10% of bugs, confidence-ranked overrides did no better than random selection on either tracker. Every one of these intervals covers 0.5 (Fig. 3). The reason is structural. The only informative confidence belongs to `choice`, the one node F does not read.

These observations led to the pre-registered tests below.

V. Pre-registered held-out test

We trained E_{dec}, an L_2-regularized logistic regression on the `score` level, the `noul` decision and the action, on the 442 development bugs. We froze it by hash, and only then collected the 500 held-out bugs (4,500 Jev calls). We pre-registered two hypotheses: E_{dec} ranks wrong actions better than low final-step confidence (HR1), and above chance on each tracker (HR2).

Both were supported. E_{dec} reached AUROC 0.763 against 0.549 for confidence, a difference of $+0.214$ with a one-sided lower bound of $+0.158$. Per tracker it reached 0.811 on Mozilla and 0.697 on the kernel (one-sided lower bounds 0.762 and 0.648). The gap over confidence was $+0.279$ [$+0.191$, $+0.374$] on Mozilla and a weaker $+0.124$ [$+0.023$, $+0.225$] on the kernel. We also checked the 70 Mozilla bugs filed in

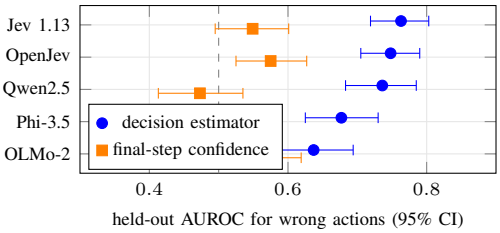


Fig. 4. For all five models, the decision estimator ranks wrong workflow actions better than the model's final-step confidence on the 500 held-out bugs. The intervals overlap only for OLMo-2.

the 12 months before sampling, which are the least likely to appear in model training data. On these, E_{dec} reached 0.847 against 0.519. The result held when every development bug linked to a held-out bug was removed and the estimator refitted (0.772 against 0.551). An independent script that shares no code with the analysis reproduced every point estimate. Its predicted failure rate matches the observed rate overall, but with only 7–8 distinct risk levels (ECE 0.18–0.20) it ranks better than it calibrates.

VI. Cross-model replication

To see whether the result is specific to Jev, we ran the same bugs, questions and adapter through four more models. OpenJev-27B [25] is an independently trained Jev-style model. The other three are small general instruct models: Qwen2.5-7B [26], OLMo-2-7B [27] and Phi-3.5-mini [28]. Jev is used zero-shot, so the fair comparison is other models used the same way, through the same questions and TypeSafe's confidence formulas. We picked small open models because they are cheap to run and fully reproducible. They are also weaker than Jev, so they produce more failures, which gives the predictors more positives to rank. The protocol had three amendments. OpenJev was added before any of its outputs existed, and OLMo-2 needed a newer `transformers` release. A readout bug found after interim results led us to re-run all three general models. The conclusions did not change.

All 8 pre-registered tests were supported (Fig. 4, Table III). Decision estimators reached AUROC 0.64–0.76, final-step confidence 0.40–0.58. The weakest case was OLMo-2 (0.637 vs. 0.551), which passed at the last Holm step. Jev's frozen estimator, never refitted, reached 0.76 [0.72, 0.80] on OpenJev's outputs, as well as OpenJev's own estimator did.

Most of this signal is simply which action was chosen. The action's error rate on development data matches the estimator for Jev (0.76) and OpenJev (0.75). It comes close for the others (0.58–0.70). For Jev the estimator adds $+0.000$ $[-0.012, +0.011]$ beyond that rate. For Qwen2.5 and OLMo-2 it adds a little ($+0.035$ and $+0.057$). Single uncertainty signals sit at chance: routing confidence at 0.48–0.52, regression confidence at 0.45–0.56 and severity entropy at 0.40–0.55. Adding them to the decision features helps only the weakest general models ($+0.054$ for Phi-3.5, $+0.078$ for OLMo-2). Part of this is built in, since the action and the target come from the same rule F. The two model families also fail differently. Jev and OpenJev pull severity to the middle level. The general models call 67–100% of bugs "blocking" whatever the true level, and they are wrong far more often (72–80% vs. 43–47%).

Because the action and the target share F, we added a stronger confidence baseline after the main results (exploratory, with the plan committed before computing). It passes the model's own probabilities through F. The risk is one minus the probability that F yields the chosen action, computed from the model's `score` distribution and `noul` probability. It did no better than final-step confidence (Jev 0.535 vs. 0.549). The action-rule score beat it for four of five models (Jev $+0.228$ $[+0.158, +0.297]$). The exception was OLMo-2 ($+0.024$ $[-0.073, +0.123]$). The decisions thus carry information beyond what the model's own probabilities provide when combined through the same rule.

In exploratory analyses, that dependence also makes the rates sensitive to the label mix (RQ2). The development set was quota-sampled by severity and the held-out set was not. Take a target that bypasses F: the severity answer being wrong. Estimators fitted on development data ranked held-out errors on it far below chance, 0.28 for Jev and OpenJev. Jev's level-1 answers were 60% wrong in the development set and 20% wrong in the natural mix. Re-weighting to the natural mix fixed this for four of five models (Table IV). For Jev it reached 0.79, against 0.61 for confidence.

Estimating the mix without labels by BBSE [16] recovered most of that gain for Jev (0.74), OpenJev (0.78) and Qwen2.5 (0.74). Its estimate of the mix, however, was wrong: for the kernel it was $[0, 1, 0]$ against a true $[0.02, 0.73, 0.25]$, and the ranking improved only because the middle level happened to dominate. Under known synthetic shifts, the estimate was biased toward the development mix. For Phi-3.5, whose severity answers are nearly all "blocking", it gave no gain at all.

VII. An independent workflow: Kubernetes

We pre-registered a replication on issue triage in the Kubernetes GitHub repository before collecting any of its data. It differs in platform, labelling process, routing (network, storage, node) and questions (bug or not, priority). The split is chronological. Development uses 500 issues from 2015–2019, and testing uses the 582 latest eligible issues from 2021–2026. Linked issues were removed and label leaks stripped from the text.

For Jev the action rule again beat confidence, 0.776 against 0.628, a paired difference of $+0.148$ $[+0.091, +0.207]$. As before, the decision estimator added little ($+0.004$ to $+0.017$). The contrast held for the any-node target, within SIG Node, and on issues triaged within 30 days. Because the two periods also differ in issue format and label mix, this is a combined test of temporal and distribution shift. A second pre-registration ran the four other models (Table V). By its fixed rule, at least three of four, the finding holds. We also defined a competence check after seeing the results and applied it to every model. Only Jev and OpenJev-27B beat chance on every step, and OpenJev is trained for Jev's interface, so these two are related systems. The three general models handled routing and the bug

TABLE III

PER-MODEL RESULTS ON THE 500 HELD-OUT BUGS. AUROC COLUMNS RANK WRONG WORKFLOW ACTIONS (95% BOOTSTRAP CI). THEY SHOW THE DECISION ESTIMATOR FITTED ON EACH MODEL'S OWN DEVELOPMENT OUTPUTS, FINAL-STEP CONFIDENCE AND THE ACTION-RULE SCORE. THE LAST COLUMN APPLIES JEV'S FROZEN ESTIMATOR UNCHANGED. COST COLUMNS ARE FROM THE PRE-REGISTERED REVIEW TEST (SECTION VIII). THE ORACLE IS AN UPPER BOUND THAT MUST SPEND ALL 50 REVIEWS.

			Held-out AUROC for wrong actions				Cost at 10% review budget			
Model	Type	Fail rate	Decision est.	Final-step conf.	Action rule	Jev est.	No review	Best review arm	Oracle	Wrong accepts
Jev	TypeSafe, hosted	0.43	0.76 [0.72, 0.80]	0.55 [0.49, 0.60]	0.76	—	658	681 (conf. product)	370	52%
OpenJev	Jev-style, 27B	0.47	0.75 [0.70, 0.79]	0.57 [0.53, 0.63]	0.75	0.76	660	674 (confidence)	441	40%
Qwen2.5	general, 7B	0.74	0.74 [0.68, 0.79]	0.47 [0.41, 0.54]	0.70	0.74	903	910 (estimator)	856	14%
Phi-3.5	general, 3.8B	0.72	0.68 [0.62, 0.73]	0.40 [0.34, 0.46]	0.69	0.70	831	890 (confidence)	890	4%
OLMo-2	general, 7B	0.80	0.64 [0.58, 0.69]	0.55 [0.48, 0.62]	0.58	0.64	893	916 (all arms tie)	916	1%

TABLE IV

EXPLORATORY: HELD-OUT AUROC OF THE DECISION ESTIMATOR ON TWO TARGETS THAT BYPASS F. IT IS FITTED ON THE QUOTA-SAMPLED DEVELOPMENT BUGS ("DEV FIT") OR RE-WEIGHTED TO THE NATURAL LABEL MIX, AND COMPARED WITH FINAL-STEP CONFIDENCE. BOLD: RE-WEIGHTED ESTIMATOR ABOVE CONFIDENCE.

	Severity wrong			Any node wrong		
Model	dev fit	re-weighted	conf.	dev fit	re-weighted	conf.
Jev	0.28	**0.79**	0.61	0.56	**0.76**	0.56
OpenJev	0.28	**0.79**	0.63	0.62	**0.81**	0.56
Qwen2.5	0.62	**0.74**	0.46	0.76	**0.73**	0.40
Phi-3.5	0.31	**0.67**	0.41	0.67	**0.77**	0.29
OLMo-2	0.50	**0.62**	0.57	0.42	0.42	0.49

TABLE V

KUBERNETES TEST SET (582 ISSUES). COLUMNS GIVE THE AUROC OF THE ACTION-RULE SCORE AND OF FINAL-STEP CONFIDENCE FOR WRONG ACTIONS, AND THEIR PAIRED DIFFERENCE (95% CI). PRIORITY IS TEST-SET PRIORITY ACCURACY (CHANCE 0.33). JEV'S ROW IS FROM THE FIRST PRE-REGISTRATION AND THE OTHER FOUR FROM THE SECOND.

Model	Rule	Conf.	Difference [95% CI]	Priority
Jev 1.13	0.776	0.628	$+0.148$ [$+0.091$, $+0.207$]	0.371
OpenJev-27B	0.796	0.569	$+0.227$ [$+0.174$, $+0.285$]	0.404
Qwen2.5-7B	0.710	0.555	$+0.155$ [$+0.095$, $+0.217$]	0.270
OLMo-2-7B	0.850	0.255	$+0.595$ [$+0.487$, $+0.700$]	0.082
Phi-3.5-mini	0.519	0.674	-0.155 [-0.284, -0.009]	0.046

question but failed on priority. Their workflows were wrong most of the time (wrong-action rates up to 95%). The rule "won" largely by flagging the one action each model nearly always chose. Phi-3.5 escalated 580 of the 582 test issues, which left the rule nothing to rank.

The label-free correction did not carry over. Re-weighting a deliberately mismatched fit by BBSE lowered its AUROC from 0.699 to 0.348 (-0.350 [-0.428, -0.271]). By its frozen rule, this test, the only pre-registered test of RQ2, is inconclusive with a negative estimate. BBSE again estimated the mix as entirely middle level, $[0, 1, 0]$. The true mix was $[0.69, 0.26, 0.05]$, so the weighting pushed the fit the wrong way. This supports our reading of the earlier gains as a coincidence of a dominant middle level.

VIII. DOES BETTER RANKING REDUCE COST?

We pre-registered the cost test before computing any cost outcome. Each model gets 50 reviews (10% of the held-out

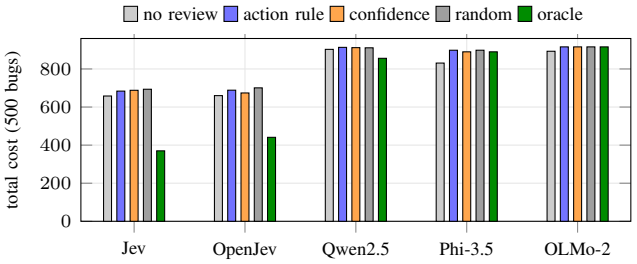


Fig. 5. Total cost at a 10% review budget under the pre-registered cost matrix. No feasible strategy beats no review. A perfect reviewer (oracle) would cut Jev's cost by 44% and OpenJev's by 33%. Point estimates are shown. Intervals for the pre-registered contrasts are given in the text.

bugs), and a review sends the case to a person, who is assumed to resolve it correctly. The primary arm ranks candidates by the expected saving of a review for their action, estimated on development data. It is compared with confidence-ranked and random review.

Two of ten pre-registered tests were supported. The action rule beat random review for Jev (-9.8 cost units [-17.4, -2.6]) and for OpenJev (-12.2 [-22.2, -3.6]). It was not shown to beat confidence for any model (Jev -4.3 [-39.6, $+36.9$]). Descriptively, with no interval pre-registered, every review arm cost more than no review at all. For Jev that was 658 with no review against 681–714 with review (Fig. 5).

The oracle bound shows where the savings are. For the Jev-style models, 40–52% of no-review cost comes from wrongly accepted serious or regression bugs. Picking the right 50 cases would save 33–44%. Those cases are only 15–16% of accepted cases, and none of the pre-registered signals could find them. Jev accepted 245 held-out cases, 38 of them wrongly. The decision estimator, final-step confidence and `score`'s top-level probability ranked those 38 at AUROC 0.46, 0.49 and 0.49. The post hoc implied-probability baseline did somewhat better (0.596 [0.508, 0.682] for Jev, 0.681 [0.589, 0.773] for OpenJev). That signal is weak, and we have not tested its value for review. For the general models the problem is different: they over-escalate, and even the oracle saves at most 5%.

The conclusion depends on the costs (Table VI). As an exploratory analysis, we re-scored the frozen rankings on cost matrices within a factor of 2 of the original. Review pays for Jev, OpenJev and Qwen2.5 when a wrong accept costs twice

TABLE VI
EXPLORATORY COST SENSITIVITY: BEST FEASIBLE ARM MINUS NO REVIEW (NEGATIVE, IN BOLD: REVIEW REDUCES COST), RANKINGS FROZEN. m SCALES WRONG-ACCEPT COSTS, h IS THE HUMAN COST PER ESCALATION AND p^* THE BREAK-EVEN PRECISION. †THE PRE-REGISTERED TABLE.

m	h	p^*	Jev	OpenJev	Qwen2.5	Phi-3.5	OLMo-2
0.5	0.5	25%	+29	+25	+21	+52	+20
0.5	1	33%	+54	+50	+46	+77	+28
0.5	2	50%	+104	+100	+96	+127	+45
1	0.5	14%	**-2**	**-11**	**-18**	+34	+14
1	1†	18%	+23	+14	+7	+59	+23
1	2	27%	+73	+64	+57	+109	+40
2	0.5	7%	**-69**	**-83**	**-97**	**-2**	+4
2	1	10%	**-44**	**-58**	**-72**	+23	+13
2	2	14%	+6	**-8**	**-22**	+73	+30

as much (for Jev, 44–69 units below no review) or a review half as much. In those cases the break-even precision falls to about the 14–16% share of wrong accepts. The ranking of the review arms was more stable than whether they beat no review: the action rule beat random for both Jev-style models in 12 of 16 cells. If reviewers are right only 80% of the time, Jev's saving at twice the wrong-accept cost falls from 44 to 26 units but does not disappear. We also tried a per-case score built to find the expensive errors. On the development mix it cut cost by 12% against no review (630 vs. 713). But the development bugs were quota-sampled, and re-weighted to the natural mix it was projected to cost more than no review. Its pre-registered feasibility check required power of at least 0.80 with at most 1,000 fresh bugs. The highest power was 0.20, so we stopped that study before collecting new data.

IX. DISCUSSION

Two features of the workflow explain most of what we found. The action rule reads only the binary and ordinal answers, while the only confidence that tracked correctness belonged to the routing step. So composed confidence mixed one useful signal with two that carried almost nothing, and it could not see the errors that mattered for the action. In addition, the action and the target come from the same rule, so an action's past error rate reflects the workflow and its label mix as much as the model. That is why it ranks failures well, and also why it moves when the mix changes.

The finding that changed the direction of the study came from an audit rather than a planned test: estimators fitted on the quota-sampled development set ranked natural-mix severity errors well below chance. Until then, the design had treated decision-conditioned error rates as properties of the model. They need to be measured under the label mix in which they will be used.

Three practical points follow for anyone building a workflow like this. Log each action's error rate on labelled data from your own setting, and recompute it whenever the mix of cases changes, instead of relying on step-level confidence. Measure how often any escalation trigger fires on pilot inputs before freezing it; our confidence trigger fired on most real inputs. Before paying for reviews, compare the precision your selector reaches on the costly errors with the break-even precision your costs imply. In our setting no pre-registered signal came close, even though a perfect selector would have cut the Jev-style models' costs by a third or more.

X. THREATS TO VALIDITY

The largest threat is construct validity. Failure means disagreeing with a target built from tracker labels, and those labels are not ground truth. Mozilla's sample excludes re-triaged bugs. Kernel severities are reporters' claims, and the kernel's middle level is a default value. In an exploratory pilot, one AI assistant chose actions for 50 held-out bugs. It agreed with the target at about chance (Cohen's κ between -0.157 and $+0.015$). With one rater we cannot tell whether the target or the judge is at fault. A two-human validation is designed but not yet run. The cost matrix is also our construct, and real review costs are unknown.

On internal validity, the cost test was pre-registered after the prediction results on the same held-out bugs were known, though before any cost was computed. The cost-sensitivity and label-shift analyses are exploratory. Old public bugs may appear in model training data, and the recent subsets we can check are small (Mozilla 54, kernel 35). The protocols were revised several times, including a readout bug in the cross-model study fixed after interim results were seen. Every revision is dated and disclosed. Our own audits caught seven defects, five of them before the data that would have exposed them existed. The confound between confidence and decisions was found only after a result.

On external validity, the study covers five models in one domain, software issue triage, across two workflows and one Jev version. On Kubernetes the general models were below chance on priority. The Kubernetes leakage read was done by an AI assistant at our direction.

XI. CONCLUSION

What a chain of typed decisions decided was a far better guide to its failures than how confident its models were. This held on 500 new bugs, for five models and on an independent Kubernetes workflow, with failure measured against a label-derived target. Most of the signal is each action's past error rate, so it has to be measured under the label mix in which it will be used. Turning a good ranking into lower cost is a separate problem. Under our cost matrix the expensive errors were wrong accepts that every pre-registered signal treated like correct ones, and review paid only when errors were made more costly or reviews cheaper.

XII. FUTURE SCOPE

The first step is to validate the target with two independent human evaluators. Beyond that, the oracle points to the real opportunity: a signal that separates costly errors within a single action. OpenJev's weak within-action severity signal (AUROC 0.68) is a place to start. The study should also be repeated outside software triage, with newer model versions, supervised baselines and measured review costs.

Data and code availability

Code, all model outputs and the analysis results are available at https://github.com/thejasrao262003/jev-compositional-uncertainty. So are the pre-registration documents, with their full commit history. The analysis code was written with AI assistance, as disclosed in the acknowledgment, and is released in full so that it can be inspected. The analyses can be re-run offline from the released outputs. A script checks every number in the full version of this paper against the result files. Re-running model calls needs a TypeSafe API key. Jev was pinned to `jev-1.13.0`, and later API versions may behave differently. OpenJev-27B was served at Hugging Face commit `b4a89f5` under vLLM 0.29.0, and Qwen2.5, Phi-3.5 and OLMo-2 under `transformers`.

Acknowledgment and AI Disclosure

Following IEEE policy on AI-generated content, we disclose that this work used Claude (Opus 5.5), a large language model by Anthropic [29], accessed through Claude Code (https://claude.com/claude-code). Claude wrote the analysis code, including the code that produces the figures and tables, and ran the analyses under our direction. The AI-assisted steps described in the text, the Kubernetes leakage read and the target-validity pilot, also used Claude. Claude prepared early drafts of the text, which we then edited and reviewed manually in full. For this version, Claude was used only to rephrase and shorten our edited text, and to revise it after a peer-review check. We directed the study, approved every pre-registration and model run, checked the AI-generated code and its outputs against the raw results, verified every citation against its bibliographic record, and take full responsibility for the content.

References

[1] T. Deußer, L. Sparrenberg, and R. Sifa, "Evaluating and benchmarking the system one model Jev," *arXiv preprint arXiv:2609.37647*, 2026.

[2] L. Meng, "How far can a commercial decision model's probabilities be trusted? A calibration audit of TypeSafe Jev against open and general-purpose classifiers," Code, data and paper, Zenodo, https://doi.org/10.5281/zenodo.22935043, 2026, version 1.0.0.

[3] S. Denney and M. DiGiuseppe, "JEV versus LLMs: Accuracy, cost and calibration on seven political science replications," *arXiv preprint arXiv:2610.06625*, 2026.

[4] M. Lin, Q. Lan, S. Guan, W. Jing, J. Fan, D. Gregg, and G. Botterweck, "Uncertainty quantification for LLM agents: A taxonomy, an evaluation protocol, and an empirical study," *arXiv preprint arXiv:2609.07395*, 2026.

[5] TypeSafe, "Confidence," TypeSafe documentation, https://docs.typesafe.ai/confidence, 2026, accessed 2026-10-16.

[6] S. Park, M. Choi, J. Kim, and M. Kang, "Risk is not review value: Wrong-answer exposure under bounded review budgets," in *SeT-LLM Workshop at KDD*, 2026, arXiv:2609.07095.

[7] J. Geuter, A. Cheng, and D. Alvarez-Melis, "Releasing the inner Jev: A simple recipe for open decision models," *arXiv preprint arXiv:2610.17205*, 2026.

[8] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, A. J. Smola, P. Bartlett, B. Schölkopf, and D. Schuurmans, Eds. MIT Press, 1999, pp. 61–74.

[9] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.

[10] R. El-Yaniv and Y. Wiener, "On the foundations of noise-free selective classification," *Journal of Machine Learning Research*, vol. 11, pp. 1605–1641, 2010.

[11] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 4878–4887.

[12] D. Madras, T. Pitassi, and R. Zemel, "Predict responsibly: Improving fairness and accuracy by learning to defer," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[13] H. Mozannar and D. Sontag, "Consistent estimators for learning to defer to an expert," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 119, 2020, pp. 7076–7087.

[14] G. Bansal, B. Nushi, E. Kamar, E. Horvitz, and D. S. Weld, "Is the most accurate AI the best teammate? optimizing AI for teamwork," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 13, 2021, pp. 11 405–11 414.

[15] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002.

[16] Z. C. Lipton, Y.-X. Wang, and A. J. Smola, "Detecting and correcting for label shift with black box predictors," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[17] J. Kittler, M. Hatef, R. P. W. Duin, and J. Matas, "On combining classifiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 20, no. 3, pp. 226–239, 1998.

[18] V. Kotte, "PASC: Pipeline-aware conformal prediction with joint coverage guarantees for multi-stage NLP and LLM pipelines," *arXiv preprint arXiv:2605.18812*, 2026.

[19] J. Zhang, S. Elluri, B. Cherukuvada, Y. Joffe, J. Sena, M. Contreras, S. Siegel, S. Nerella, C. Price, and P. Rashidi, "Auditing multimodal LLM raters: Central tendency bias in clinical ordinal scoring," *arXiv preprint arXiv:2605.16386*, 2026.

[20] M. J. Zellinger, R. Liu, and M. Thomson, "Cost-saving LLM cascades with early abstention," *arXiv preprint arXiv:2502.09054*, 2025.

[21] D. Bouchard, "Is escalation worth it? On the depth of LLM cascades," *arXiv preprint arXiv:2605.06350*, 2026, v2; v1 was titled "Is Escalation Worth It? A Decision-Theoretic Characterization of LLM Cascades".

[22] J. Jung, F. Brahman, and Y. Choi, "Trust or escalate: LLM judges with provable guarantees for human agreement," in *International Conference on Learning Representations (ICLR)*, 2025.

[23] V. Cortellessa and V. Grassi, "A modeling approach to analyze the impact of error propagation on reliability of component-based systems," in *Component-Based Software Engineering (CBSE 2007)*, ser. LNCS, vol. 4608. Springer, 2007, pp. 140–156.

[24] A. Lamkanfi, S. Demeyer, E. Giger, and B. Goethals, "Predicting the severity of a reported bug," in *7th IEEE Working Conference on Mining Software Repositories (MSR)*, 2010, pp. 1–10.

[25] OpenJev, "OpenJev: an open-weights decision model," https://huggingface.co/openjev/openjev, 2026, Hugging Face commit b4a89f5; CC BY-NC 4.0 weights; states it is independent of TypeSafe. Not the same project as other repositories named OpenJev. Accessed 2026-10-17.

[26] Qwen, A. Yang *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[27] Team OLMo, P. Walsh, L. Soldaini *et al.*, "2 OLMo 2 furious," *arXiv preprint arXiv:2501.00656*, 2024.

[28] M. Abdin, J. Aneja, H. Awadalla *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," *arXiv preprint arXiv:2404.14219*, 2024.

[29] Anthropic, "Claude (Opus 5.5) [Large language model]," https://claude.ai/, 2026, accessed through Claude Code, https://claude.com/claude-code.